Order Documentation Tracker Technical Report

Architecture, database, API and technology reference

Version	2.0 (Vercel edition)
Stack	Vercel, Node 22, PostgreSQL (pg), Vercel Blob, React 18
Source size	lib/app.js 317 lines, public/index.html 356 lines
Prepared	October 2026

Contents

1. Introduction **4**

1.1 Feature summary . . . 4

2. Technology stack **5**

3. System architecture **5**

3.1 Request lifecycle . . . 6

3.2 Why this shape . . . 6

4. Folder structure **6**

5. Deployment and configuration **7**

5.1 vercel.json . . . 7

5.2 Environment variables . . . 7

5.3 Local development . . . 7

5.4 Deployment procedure (summary) . . . 7

6. Database **8**

6.1 Engine and access . . . 8

6.2 Schema . . . 8

6.3 JSON documents . . . 9

6.4 ORM . . . 9

6.5 Concurrency and integrity . . . 9

6.6 Query inventory . . . 10

7. Backend **10**

7.1 Module layout (lib/app.js) . . . 10

7.2 Authentication and sessions . . . 10

7.3 Authorisation matrix . . . 11

7.4 Validation (clean) . . . 11

7.5 Audit log (diffLog) . . . 11

7.6 Error handling . . . 11

7.7 Attachments - detail . . . 11

8. API reference **11**

8.1 Status codes . . . 12

8.2 Examples . . . 12

9. Frontend **13**

9.1 Structure . . . 13

9.2 Component tree . . . 13

9.3 State management . . . 13

9.4 Browser APIs and presentation 14

10. Security **14**

11. Limitations and operational notes **14**

1. Introduction

The **Order Documentation Tracker** is a web application for a valve manufacturer. For every customer purchase order (PO) it tracks the documents the manufacturer must submit to the customer (drawings, material certificates, test reports, data book) against weekly milestones counted from the PO receipt date. A dashboard shows which orders are on track, due soon, overdue or complete. It works on desktop, tablet and phone.

This document is the technical reference for the version deployed on **Vercel**. It describes the architecture, folder structure, database, API endpoints, backend, frontend and every technology the system relies on. A second document, the Code Intuition Report, explains the code flow function by function.

1.1 Feature summary

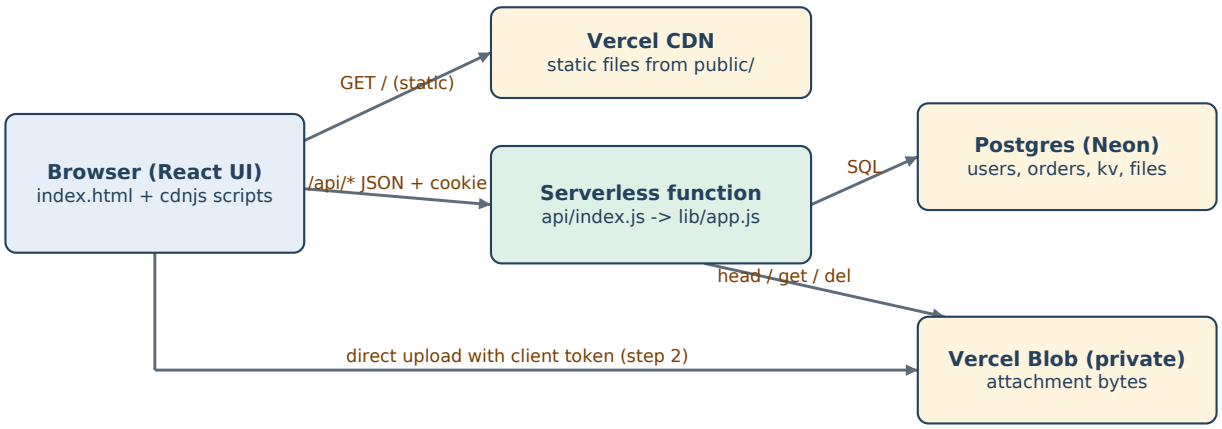
- **Orders:** create, edit, delete; PO date, order number, customer, optional e-mail(s), estimated delivery, manufacturer reference, "documentation due by week", unlimited custom extra fields, and per-order renaming/removal of built-in fields.
- **Milestones:** each order has weekly milestones (week number, title, reminder lead time in days). Milestones can be passed, un-passed and annotated with remarks.
- **Dashboard:** status cards (all / completed / on track / due soon / overdue), orders-per-year bars, open orders by current milestone, search across all years, filters, CSV export. The year runs September to August.
- **Attachments:** files (PO e-mail, documents) are uploaded straight from the browser to a private Vercel Blob store and downloaded through the app.
- **Roles:** viewer (read only), editor (orders, milestones, files), admin (plus settings, users, deleting orders). The dashboard can be public or login-only.
- **Audit trail:** every create, edit, pass, remark, reminder change and file change is logged on the order with user and time.
- **E-mail reminder:** a button opens the user's mail client with a prefilled status message to all customer addresses.
- **Print** view and **CSV/Excel export**.

2. Technology stack

Layer	Technology	Version	Role in the system
Hosting / runtime	Vercel (Serverless Functions, Node.js runtime, static CDN)	Node 22.x	Serves `public/` from the CDN and runs `api/index.js` as one function for every `/api/*` request.
Language (backend)	JavaScript (CommonJS) on Node.js	22.x	All server logic in `lib/app.js`. No framework (no Express); the handler uses the plain `(req, res)` Node signature.
Database	PostgreSQL (Neon, via Vercel Marketplace)	any Postgres	Stores users, orders, settings, file records, login-failure counters.
DB driver	`pg` (node-postgres)	^8.13.0	Connection pool and parameterised queries. **No ORM** (see section 6.4).
File storage	Vercel Blob (private store)	SDK `@vercel/blob` ^2.8.1	Holds uploaded attachments. Browser uploads directly with a short-lived token issued by the server.
Cryptography	Node `crypto` module	built in	scrypt password hashing, HMAC-SHA256 signed session cookies, random IDs, timing-safe comparison.
Frontend framework	React (UMD build, hooks)	18.2.0	Whole UI in one HTML file. State with `useState`, `useEffect`, `useMemo`, `useCallback`.
JSX compiler	Babel Standalone (in the browser)	7.23.5	Compiles the `<script type="text/babel">` block on page load. No build step.
Script delivery	cdnjs.cloudflare.com	-	Hosts React, ReactDOM and Babel.
Upload client	`@vercel/blob/client` via esm.sh	2.8.1	Loaded with a dynamic `import()` only when a file is uploaded.
Styling	Plain CSS with custom properties	-	Light/dark theme through `prefers-color-scheme`, print stylesheet, responsive grid.
Source control / CI	GitHub + Vercel Git integration	-	Every commit to the repository triggers a new deployment.

3. System architecture

The system has three tiers. The **browser** holds the entire user interface and calls a JSON API. A single **serverless function** implements the API, authentication and all business rules. **Postgres** stores structured data and **Vercel Blob** stores file contents. Static files never touch the function: Vercel serves `public/` from its CDN.



3.1 Request lifecycle

- The browser requests `/`; the CDN returns `public/index.html`, which loads React, ReactDOM and Babel from cdnjs and compiles the UI.
- The UI calls `GET /api/state`. `vercel.json` rewrites `/api/(.*)` to `/api`, so `api/index.js` runs and hands the request to `lib/app.js`.
- On the first request of a cold function instance `init()` creates missing tables, loads the signing secret and bootstraps the admin user. Later requests skip this (the promise is cached).
- `api()` identifies the user from the signed cookie, loads the settings, enforces the CSRF header on writes, then dispatches on method and path.
- Handlers run parameterised SQL through the `pg` pool and reply with JSON using `send()`. Errors are thrown as objects with a `status` and turned into `{error: message}` responses by the top-level handler.
- File **uploads** bypass the function body: the function only issues a token (`POST /api/blob`); the browser then PUTs the bytes to Blob and finally registers the file (`POST /api/orders/:id/files`).
- File **downloads** go through the function (`GET /api/files/:id`), which checks access and streams the private blob to the client.

3.2 Why this shape

Vercel functions are stateless and have no persistent disk, so nothing may be kept in memory or on the local file system between requests. Consequently: data lives in Postgres, files in Blob, the login-failure counter in a table, and order updates use optimistic concurrency in SQL. Vercel also limits request bodies to 4.5 MB, which is why files are uploaded directly to Blob.

4. Folder structure

```
valve-tracker/
|-- api/
|   `-- index.js          Vercel entry point (3 lines): re-exports lib/app.js
|-- lib/
|   `-- app.js            ALL backend logic: DB, auth, validation, endpoints (317 lines)
|-- public/
|   `-- index.html        ALL frontend: HTML + CSS + React/JSX (356 lines)
|-- server.js             Local development server only (not used on Vercel)
|-- reset-admin.js        CLI tool: create/reset an admin account directly in Postgres
|-- vercel.json           Routing, output directory, function timeout, security headers
|-- package.json          Dependencies (pg, @vercel/blob) and Node version
|-- .env.example          Template of environment variables
|-- .gitignore            Ignores node_modules, .env, .env.local, .vercel
`-- README.md             Short operator guide
```

File	Purpose	Runs where
`api/index.js`	The only serverless function. Vercel maps the file `api/index.js` to the URL `/api`; the rewrite in `vercel.json` sends every `/api/*` URL to it.	Vercel (Node)
`lib/app.js`	Exports `handler(req, res)`. Contains the pool, schema bootstrap, session/password helpers, validators, audit-log diffing, optimistic-lock helpers and the whole endpoint router.	Vercel (Node) / local
`public/index.html`	Single-page UI. CSS in a `<style>` block; React components in a `<script type="text/babel">` block.	Browser
`server.js`	Minimal `http` server: routes `/api/*` to the same handler and serves `public/` files, so the app can run locally exactly like on Vercel.	Local only
`reset-admin.js`	Hashes a password with scrypt and updates or inserts an admin row. Used for lost-password recovery.	Operator machine
`vercel.json`	Deployment configuration (section 5).	Vercel build
`package.json`	Declares `pg` and `@vercel/blob`; `engines.node` is `22.x`.	Vercel build

5. Deployment and configuration

5.1 vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": null,
  "outputDirectory": "public",
  "rewrites": [{ "source": "/api/(.*)", "destination": "/api" }],
  "functions": { "api/index.js": { "maxDuration": 60 } },
  "headers": [
    { "source": "/(.*)", "headers": [
      { "key": "X-Content-Type-Options", "value": "nosniff" },
      { "key": "X-Frame-Options", "value": "DENY" },
      { "key": "Referrer-Policy", "value": "same-origin" } ] }
  ]
}
```

- `buildCommand: null` - nothing to compile; Vercel installs dependencies and deploys.
- `outputDirectory: "public"` - the CDN serves this folder at the site root (`/` returns `index.html`).
- `rewrites` - every `/api/...` URL is internally rewritten to the function at `/api`; the original URL path is kept in `req.url`, which the router reads.
- `functions.maxDuration: 60` - a request may run up to 60 seconds (needed for large downloads streaming through the function).
- `headers` - adds `X-Content-Type-Options`, `X-Frame-Options: DENY` (anti-clickjacking) and `Referrer-Policy` to every response.

5.2 Environment variables

Variable	Source	Required	Meaning
`DATABASE_URL` (or `POSTGRES_URL`)	Added by the Neon integration	Yes	Postgres connection string. If missing the app logs `DATABASE_URL is not set` and queries fail.
`BLOB_READ_WRITE_TOKEN`	Added by the Blob store connection	Yes (uploads)	Server credential for Blob `head`, `get`, `del` and for minting client upload tokens.
`ADMIN_PASSWORD`	You	First start	Password of the first admin. Used only while the `users` table is empty. If absent a random one is generated and logged once.
`ADMIN_USER`	You	No (admin)	Username of the first admin.
`SECRET`	You	Recommend ed	HMAC key for session cookies. If unset, a random key is created and stored in the `kv` table.
`SESSION_DAYS`	You	No (7)	Session cookie lifetime in days.
`MAX_UPLOAD_MB`	You	No (100)	Maximum attachment size enforced in the upload token (`maximumSizeInBytes`).
`COOKIE_SECURE`	You	No	Set to `1` to force the Secure cookie flag. It is also set automatically when `x-forwarded-proto` is `https`.
`VERCEL`, `TRUST_PROXY`	Platform / you	No	When `VERCEL=1` (set by Vercel) or `TRUST_PROXY=1`, the client IP is taken from `x-forwarded-for` for login throttling.

5.3 Local development

```
npm install
# .env.local contains DATABASE_URL and BLOB_READ_WRITE_TOKEN
node --env-file=.env.local server.js        # http://localhost:3000
node --env-file=.env.local reset-admin.js admin NewPassword123   # recover admin access
```

5.4 Deployment procedure (summary)

- Push the repository to GitHub and import it into Vercel (Framework preset: Other).

- Add a Neon Postgres database and a **private** Blob store under the project's Storage tab and connect both to all environments.
- Set `ADMIN_PASSWORD` and `SECRET`, then redeploy so that new variables take effect.
- Open the site and sign in as `admin`. Tables are created on the first request.

6. Database

6.1 Engine and access

PostgreSQL, accessed through a `pg.Pool` (maximum 3 connections per function instance, idle timeout 10 s). Two thin helpers wrap the pool: `q(sql, params)` returns all rows and `q1(sql, params)` returns the first row or `undefined`. Every statement uses positional parameters (`$1`, `$2`, ...), so user input never becomes part of SQL text.

6.2 Schema

The schema is created idempotently by `init()` using `CREATE TABLE IF NOT EXISTS`. There is no separate migration tool; a change to the schema means editing `init()` (additive changes such as a new column can be added with `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`).

users

Column	Type	Notes
`id`	`SERIAL PRIMARY KEY`	Numeric id; stored in the session cookie.
`username`	`TEXT NOT NULL`	Unique case-insensitively through the index `users_username_ci` on `LOWER(username)`.
`name`	`TEXT`	Display name (used in the audit log).
`role`	`TEXT`	`admin`, `editor` or `viewer` (validated in code).
`pw`	`TEXT`	`salt:hash` - 16-byte random salt (hex) and 32-byte scrypt hash (hex).
`tv`	`INTEGER DEFAULT 0`	"Token version". Part of the session; incremented on password change/reset and on deactivation to invalidate existing sessions.
`active`	`INTEGER DEFAULT 1`	1 = active, 0 = disabled.

orders

Column	Type	Notes
`id`	`TEXT PRIMARY KEY`	12 hex characters from `crypto.randomBytes(6)`.
`data`	`TEXT`	The whole order as a JSON string (structure in 6.3).
`rev`	`INTEGER DEFAULT 1`	Revision counter used for optimistic locking; incremented on every successful write.

files

Column	Type	Notes
`id`	`TEXT PRIMARY KEY`	24 hex characters. Used in the download URL `/api/files/:id`.
`oid`	`TEXT`	Id of the owning order (logical foreign key; not enforced by a constraint).
`name / type`	`TEXT`	Original file name and MIME type.
`size`	`BIGINT`	Size in bytes, read from Blob after upload.
`uploader / ts`	`TEXT`	User display name and `YYYY-MM-DD HH:MM` (UTC).
`path`	`TEXT`	Blob pathname (`orders/<orderId>/<name>-<random>`); used to read and delete the blob.

kv

Column	Type	Notes
k	TEXT PRIMARY KEY	Key. Two keys are used: cfg (application settings) and secret (generated cookie-signing key).
v	TEXT	JSON-encoded value.

login_fails

Column	Type	Notes
ip	TEXT PRIMARY KEY	Client IP address.
n	INTEGER	Number of consecutive failed logins.
t	BIGINT	Epoch milliseconds of the last failure. 8 or more failures within 15 minutes yield HTTP 429.

6.3 JSON documents

Orders and settings are stored as JSON text rather than as many relational columns. This keeps the flexible parts (renamable fields, custom fields, variable numbers of milestones) simple, at the cost of not being able to filter orders in SQL; the application loads all orders and filters in the browser.

Order document (column orders.data)

```
{
  "date": "2026-09-10",            // PO receipt date (required)
  "orderNo": "PO-1",               // required, max 80
  "customer": "ACME",              // required, max 200
  "email": "a@x.com, b@y.com",     // up to 20 addresses, validated
  "est": "2026-12-01", "mref": "MR-77",
  "weeks": 12,                      // documentation due by week (1-520)
  "extra":  { "Project no": "P-9" },          // custom fields (max 40)
  "labels": { "customer": "Buyer" },          // renamed built-in fields
  "hide":   ["mref"],                          // removed optional fields
  "ms": [ { "w": 2, "t": "Drawings", "r": 7, "done": "2026-09-20", "note": "..." } ],
  "files": [ { "id": "ab12...", "name": "po.pdf", "size": 120394 } ],
  "log":   [ { "who": "Administrator", "at": "2026-09-10 08:15", "what": "Order created" } ]
}
```

Settings document (kv key cfg)

```
{ "app": "Order Documentation Tracker", "mfr": "Your Valve Company", "weeks": 12,
  "ac": "#24415e", "public": true, "fields": ["Project no"],
  "ms": [ { "w": 2, "t": "Drawings & GA submitted", "r": 7 }, ... ] }
```

6.4 ORM

The application does not use an ORM (no Prisma, Sequelize, TypeORM or Drizzle). Data access is hand-written SQL through `pg`. Reasons: the schema has five small tables; most data is stored as JSON documents so an object-relational mapping would add little; and avoiding a dependency keeps cold starts short. The trade-offs are manual schema management in `init()` and no type generation.

6.5 Concurrency and integrity

- **Optimistic locking.** Updates run `UPDATE orders SET data=$2, rev=rev+1 WHERE id=$1 AND rev=$3`. If the row count is 0 someone else changed the order first: the API answers 409 and the UI reloads the latest version.
- **Retry helper.** `mutate()` (used for attachments, which the user did not edit through the form) re-reads and retries up to 6 times before giving up with 409.
- **Idempotent bootstrap.** `INSERT ... ON CONFLICT DO NOTHING` makes simultaneous cold starts safe when creating the secret and the first admin.
- **Unique usernames.** The case-insensitive unique index plus `ON CONFLICT DO NOTHING` returns "Username already exists" without race conditions.

- **Last-admin protection.** `lastAdmin(id)` prevents deleting, demoting or deactivating the only active admin.

6.6 Query inventory

Purpose	SQL (abridged)
Load all orders	`SELECT * FROM orders`
Load one order	`SELECT * FROM orders WHERE id=$1`
Create order	`INSERT INTO orders(id,data,rev) VALUES($1,$2,1)`
Update order (CAS)	`UPDATE orders SET data=$2, rev=rev+1 WHERE id=$1 AND rev=$3`
Delete order and files	`DELETE FROM files WHERE oid=$1; DELETE FROM orders WHERE id=$1`
Find user at login	`SELECT * FROM users WHERE LOWER(username)=LOWER($1)`
Session lookup	`SELECT * FROM users WHERE id=$1`
Throttle login	`SELECT/INSERT ... ON CONFLICT(ip) DO UPDATE / DELETE FROM login_fails`
Settings	`SELECT v FROM kv WHERE k=$1; INSERT ... ON CONFLICT(k) DO UPDATE`
File record	`INSERT INTO files(...); SELECT * FROM files WHERE id=$1`

7. Backend

7.1 Module layout (lib/app.js)

Section	Contents
Imports and constants	`crypto`, `pg`, `@vercel/blob`, limits (`MAXUP`, `SESSION_DAYS`), connection pool, `q`/`q1`, `E` (error factory), `now`, `uid`.
Schema + bootstrap	`init()`; cached in the `ready` promise; creates tables, index, secret and first admin.
Settings	`kvGet`, `kvSet`, defaults `DEF`, `getCfg` (defaults merged with stored values).
Passwords & sessions	`hashPw`, `checkPw`, `sign`, `unsign`, `cookies`, `setCookie`, `userFrom`, `issue`, `ipOf`.
Validation & audit	`S`, `D`, `R`, `emails`, `clean`, `diffLog`; `getOrder`, `strip`, `casOrder`, `mutate`.
HTTP helpers	`send`, `readRaw`, `readJson`, `lastAdmin`, `safeName`.
Router	`api()` - one function with an `if` per endpoint.
Export	`module.exports = handler` which calls `init()` and `api()` inside a try/catch.

7.2 Authentication and sessions

- **Login** (`POST /api/login`) looks the user up case-insensitively, checks the scrypt hash with `crypto.timingSafeEqual`, then sets the cookie `vt`.
- **Cookie value** = `base64url(JSON{u: userId, tv: tokenVersion, e: expiryMs}) + "." + HMAC-SHA256`. Flags: `HttpOnly; SameSite=Lax; Path=/; Max-Age; Secure` (when HTTPS).
- **Validation** on each request (`userFrom`): signature valid, not expired, user exists, active, and `tv` equals the user's current `tv`. Changing/resetting a password or deactivating the user increments `tv`, which instantly invalidates old cookies.
- **Throttling:** 8 failures within 15 minutes from the same IP gives HTTP 429. The counter is in Postgres so it works across serverless instances.

7.3 Authorisation matrix

Capability	Anonymous	Viewer	Editor	Admin
Read orders, download files	Only if "open dashboard" is on	Yes	Yes	Yes
Change own password	No	Yes	Yes	Yes
Create / edit orders, pass milestones, upload / remove files	No	No	Yes	Yes
Delete orders	No	No	No	Yes
Edit settings, manage users	No	No	No	Yes

The check is the `need(role)` closure inside `api()`: `need()` = logged in, `need('editor')` = not viewer, `need('admin')` = admin only. `canRead()` allows anonymous reads when `cfg.public` is true.

7.4 Validation (clean)

`clean(d, old)` is the single gate for order data. It rejects a missing date, order number or customer (HTTP 400), trims and truncates every string, drops unknown label keys and non-hideable hide keys, clamps numbers (week 0-520, reminder 0-365, weeks 1-520), limits the arrays (60 milestones, 40 extra fields) and validates e-mail addresses. It always takes `files` and `log` from the stored order, never from the client, so attachments and history cannot be forged through the order form.

7.5 Audit log (diffLog)

On every update the server compares the old and new order and appends entries such as "Order details edited", "W4 Material certificates: Passed", "Pass undone", "remarks updated", "reminder set to 10 days before due", "milestone added/removed". Milestones are matched by the key `week|title`. The log is capped to the latest 500 entries.

7.6 Error handling

Handlers throw `E(status, message)`. The exported handler catches everything: errors with a `status` become `{error: message}` with that status; any other error is logged and returned as HTTP 500 `{error: "Server error"}`. If headers were already sent (streaming download) the response is simply ended.

7.7 Attachments - detail

- **Token** (`POST /api/blob`): requires editor. `handleUpload()` from `@vercel/blob/client` validates the event; `onBeforeGenerateToken` checks the pathname starts with `orders/<oid>/`, that the order exists, and returns `maximumSizeInBytes` and `addRandomSuffix`.
- **Register** (`POST /api/orders/:id/files`): re-checks the path prefix, calls `blob.head()` to confirm the blob exists and read its real size and type, refuses duplicates, inserts into `files`, then adds the entry and a log line to the order with `mutate()`.
- **Download** (`GET /api/files/:id`): checks read access, calls `blob.get(path, {access:"private"})`, and streams the body. Only PDF, PNG, JPEG, GIF and WebP are served inline; everything else is forced to download as `application/octet-stream`. `Content-Security-Policy: sandbox` and `nosniff` are added.
- **Delete:** removes the entry from the order, deletes the blob (best effort) and the `files` row. Deleting an order deletes all its blobs.

8. API reference

All endpoints live under `/api/` and exchange JSON. **Every non-GET request must carry the header** `x-req: 1` (CSRF defence: a cross-site form post cannot add custom headers). Authentication is the `vt` cookie sent automatically by the browser. Unknown routes return 404 `{error:"Not found"}`.

Method	Path	Auth	Request body / query	Success response	Errors
GET	/api/state	Public if open dashboard, else login	-	`{me, cfg, orders[]}`; when locked: 401 `{error:"login", brand}`	401
POST	/api/login	None	`{username, password}`	`{me}` + Set-Cookie `vt`	401, 429
POST	/api/logout	None	-	`{}` and clears cookie	403
POST	/api/password	Logged in	`{old, new}` (new >= 8 chars)	`{}` + new cookie	400, 401
POST	/api/orders	Editor	`{data: order}`	`{id}`	400, 401, 403
PUT	/api/orders/:id	Editor	`{rev, data: order}`	`{rev: newRev}`	400, 404, 409
DELETE	/api/orders/:id	Admin	-	`{}`	401, 403
POST	/api/blob	Editor	Vercel Blob upload event (token request)	`{type, clientToken}`	400, 401, 404
POST	/api/orders/:id/files	Editor	`{name, type, pathname}`	`{id}`	400, 404, 409
GET	/api/files/:id	Public if open, else login	-	Streamed file bytes	401, 404
DELETE	/api/files/:id	Editor	-	`{}`	401, 403, 404, 409
PUT	/api/cfg	Admin	`{app, mfr, weeks, public, ac, fields[], ms[]}`	`{}`	400, 401, 403
GET	/api/users	Admin	-	`{users[]}`	401, 403
POST	/api/users	Admin	`{username, name, password, role}`	`{}`	400, 409
PUT	/api/users/:id	Admin	`{role?, active?, name?, password?}`	`{}`	400, 404
DELETE	/api/users/:id	Admin	-	`{}`	400, 404

8.1 Status codes

Code	Meaning in this app
400	Validation failed (missing required fields, bad e-mail, short password, bad path, invalid JSON, last-admin rule).
401	Not logged in or session invalid; `/api/state` returns `{error:"login", brand}` so the UI can show the sign-in screen with branding.
403	Missing `x-req` header or insufficient role.
404	Order, file, user or route not found.
409	Revision conflict (order changed by someone else), username exists, or file already attached.
413	Request body too large (only when reading the raw stream locally).
429	Too many failed logins from this IP.
500	Unexpected server error; details are only in the Vercel runtime logs.

8.2 Examples

Login

```
POST /api/login        x-req: 1
{"username":"admin","password":"********"}
-> 200 {"me":{"id":1,"username":"admin","name":"Administrator","role":"admin","active":true}}
   Set-Cookie: vt=<payload>.<signature>; HttpOnly; SameSite=Lax; Path=/; Max-Age=604800; Secure
```

Update an order (optimistic lock)

```
PUT /api/orders/f929a1e36f85     x-req: 1
{"rev":1,"data":{"date":"2026-09-10","orderNo":"PO-1","customer":"ACME","ms":[...]}}
-> 200 {"rev":2}
-> 409 {"error":"Order was changed by someone else"}     (when rev is stale)
```

Three-step upload

```
1) browser -> POST /api/blob          (Vercel Blob SDK asks for a token; payload carries {oid})
   server  -> {"type":"blob.generate-client-token","clientToken":"..."}
2) browser -> Vercel Blob directly    (multipart for files > 8 MB, private access)
3) browser -> POST /api/orders/<oid>/files   {"name":"po.pdf","type":"application/pdf","pathname":"orders/<oid>/po-AbC123
   server  -> {"id":"<fileId>"}
```

9. Frontend

9.1 Structure

The frontend is one file, `public/index.html` (356 lines): a `<style>` block, a `<div id="root">` and a `<script type="text/babel">` block. React, ReactDOM and Babel Standalone are loaded as UMD scripts from cdnjs. Babel compiles the JSX in the browser, so there is no bundler, no `node_modules` for the UI and no build step. The page renders with `ReactDOM.createRoot(...).render(<App/>)`.

9.2 Component tree

```
App                       tabs, header, modals, global run() error handler
|-- Login                 sign-in form (full page when locked, modal otherwise)
|-- Dashboard             year range, cards, stage bars, search/filter/export, order list
|   `-- Track             coloured milestone strip for one order
|-- OrderForm             create/edit order (also used inside a modal for editing)
|   |-- LabelEdit         inline-renamable field label
|   |-- EmailsInput       e-mail chips
|   `-- MsEdit            one milestone row (week, title, reminder days)
|-- Detail (modal)        one order: milestones, files, history, actions
|   `-- Track
|-- Settings              app settings + default milestones
|   |-- MsEdit
|   `-- Users             user administration table
`-- Account (modal)       change own password
```

Component / helper	Responsibility
`api(method, url, body)`	Wrapper around `fetch`: same-origin credentials, `x-req: 1` header, JSON in/out, throws an `Error` carrying `status` and `body` for non-2xx responses.
`useApp()`	Custom hook holding global state `{ready, locked, me, cfg, orders, brand}`; loads `/api/state` on mount and every 20 seconds while the tab is visible; exposes `load()`.
`ost(order)`, `cls(order)`	Compute milestone states (done / late / open), the current milestone, "due soon" and overall colour. Pure functions used everywhere.
`Dashboard`	Derives all statistics with `useMemo`; filtering, searching and CSV export happen client-side.
`OrderForm`	Local copy of the order in state; field renaming/removal, custom fields, milestone list; sends POST or PUT.
`Detail`	Read/act view: pass/undo milestones, remarks (saved on blur), upload/remove files, e-mail reminder, print, delete.
`Settings`, `Users`	Admin-only configuration and user management.
`App`	Owns tab state and modals; `run(fn)` executes an action, shows errors (special text for 409), then reloads data.

9.3 State management

There is no Redux or context. Server state lives in `useApp()` and is passed down as the prop `S`; `S.run(fn)` is the universal "perform a write, then refresh" function. Each form keeps its own local state. After every write, and every 20 seconds, the whole state (settings plus all orders) is re-fetched, which keeps several users in sync without WebSockets.

9.4 Browser APIs and presentation

- `fetch`, `URL.createObjectURL(Blob)` (CSV download), `window.print()`, `mailto:` links, `document.hidden` (pause polling), dynamic `import()` (Blob client).
- CSS custom properties define the palette; `@media (prefers-color-scheme: dark)` swaps it. The theme colour from settings overrides `--ac` at runtime.
- Responsive: CSS grid with `auto-fit`/`minmax`, flex wrapping, `viewport-fit=cover` with safe-area padding for phones.
- `@media print` hides navigation and buttons so an order can be printed as a status sheet.

10. Security

Topic	Measure
Passwords	scrypt with random 16-byte salt, 32-byte key; compared with `timingSafeEqual`; minimum 8 characters.
Sessions	HMAC-SHA256 signed, HttpOnly, SameSite=Lax, Secure on HTTPS, expiring; revocable through the token version `tv`.
CSRF	All non-GET calls require `x-req: 1`; combined with SameSite=Lax cookies.
SQL injection	Only parameterised queries.
XSS	React escapes all rendered text; the API never returns HTML; downloads are sandboxed (`CSP: sandbox`) and non-image/PDF types are forced to download.
Brute force	Login throttling per IP (8 failures / 15 min) stored in Postgres.
Clickjacking	`X-Frame-Options: DENY`.
Upload control	Token issued only to editors, restricted to `orders/<existing order>/` paths, size-limited; file registration re-validates the path and reads real size from Blob; blobs are private.
Least privilege	Role checks on every endpoint; last active admin cannot be removed; client can never set `files` or `log`.
Secrets	Only in Vercel environment variables; `.env` and `.env.local` are git-ignored.

11. Limitations and operational notes

- **Whole-dataset loading.** `/api/state` returns every order and the UI polls every 20 s per open tab. This is fine for hundreds or a few thousand orders; for much larger volumes add pagination or server-side filtering.
- **In-browser Babel.** The console warning "You are using the in-browser Babel transformer" is expected. It costs some first-load time; precompiling the JSX would remove it.
- **External runtime dependencies.** The UI depends on cdnjs (React, Babel) and, for uploads only, esm.sh. If those hosts are blocked, the UI or uploads fail. They can be vendored into `public/vendor/`.
- **Orphaned blobs.** If an upload finishes but the registration call fails (for example the user closes the tab), the blob remains in the store without a database record. Periodic clean-up can compare Blob contents with the `files` table.
- **Time zones.** Server timestamps are UTC; "today" for due-date logic is computed in the viewer's browser using local time.
- **Idle database.** Neon free databases may suspend when idle, so the first request after a pause can be slower.
- **Public dashboard.** With "open dashboard" on, anyone with the URL can view orders and download attachments. Turn it off in Settings if documents are confidential.
- **E-mail.** Reminders open the user's mail client; the server does not send e-mail.
- **Schema changes.** There is no migration framework; edit `init()` and redeploy.
- **Backups.** Use Neon's backup/restore for data; the Blob store holds files separately.